INSTITUTO MAUÁ DE TECNOLOGIA

MAUÁ

Aula 05 – Interrupção de Hardware

ECA409 – Microcontroladores e Sistemas Microcontrolados

Autor: Prof. Me. Rodrigo França

Uso de Interrupções no RP2040

INSTITUTO MAUÁ DE TECNOLOGIA

MAUÁ

- **Objetivos da Aula:**
  - Explicar o conceito de interrupções e sua relevância.
  - Apresentar o mecanismo de interrupções na arquitetura ARM (Cortex-M0+).
  - Mostrar como configurar e usar o NVIC e ISRs.
  - Explorar a lista de interrupções do RP2040.
  - Demonstrar interrupção de GPIO e modos de baixo consumo.
  - Desenvolver exemplos práticos com botão e *sleep mode*.

O que são interrupções?

- **Definição:**
  - Interrupções são sinais de hardware ou software que interrompem o fluxo normal de execução do processador para lidar com um evento específico.

- **Finalidade:**
  - Tornar o sistema **reativo**, respondendo rapidamente a eventos externos ou internos.
  - **Economizar processamento:** evita ficar verificando (*polling*) continuamente um estado (ex.: um botão).
  - Permitir que o microcontrolador entre em modo de **baixo consumo** e acorde só quando necessário.

- **Exemplos de uso:**
  - Botões, sensores, *timers*, eventos de comunicação (UART, USB, SPI, I2C etc.).
  - Alertas de falhas (*watchdog*, *brown-out*, etc.).

Por que usar interrupções?

- **Eficiência:**
  - Evita uso de "*while-loop*" constante para verificar periféricos.
  - Reduz consumo de energia.
- **Responsividade:**
  - Permite reação imediata a eventos críticos, como interrupção de timer ou pino de emergência.
- **Organização de código:**
  - Separa o tratamento de eventos em rotinas claras (ISRs).
  - Facilita manter o loop principal para lógica de alto nível ou tarefas menos críticas.
- **Exemplo prático:**
  - Sistema de leitura de *encoder* que dispara interrupções a cada pulso. Sem interrupções, a CPU teria de fazer *polling* contínuo, correndo o risco de perder pulsos rápidos.

Interrupções no RP2040 e Arquitetura ARM Cortex-M0+

- **RP2040:**
  - Microcontrolador da Raspberry Pi Foundation.
  - **Dual-core ARM Cortex-M0+** a até 133 MHz.
  - Recursos de GPIO versáteis + PIO programável.

- **Cortex-M0+:**
  - Arquitetura ARM otimizada para baixo consumo e simplicidade.
  - **Vetor de interrupções** : uma tabela que aponta para as rotinas de tratamento (ISRs).
  - **NVIC** (*Nested Vectored Interrupt Controller*) gerencia as prioridades e o despacho das interrupções.

- **Destaques no RP2040:**
  - Múltiplas fontes de interrupção (GPIO, timers, UART, USB, I2C, SPI, PIO, etc.).
  - Modos de baixo consumo personalizáveis, facilitando aplicações em IoT.

Controlador NVIC (*Nested Vectored Interrupt Controller*)

- **Papel do NVIC:**
  - Gerenciar interrupções de forma **aninhada** (*nested*), permitindo interrupções com prioridades mais altas interromperem ISRs de menor prioridade.
  - **Habilitar/Desabilitar** interrupções.
  - Configurar **prioridades**.

- **Funcionalidades principais:**
  1. **Registro de ativação (Enable/Disable Registers):** para cada fonte de interrupção.
  2. **Registro de prioridade (Priority Registers):** define a prioridade de cada vetor de interrupção.
  3. **Manipulação de Pending & Active:** checa se a interrupção está pendente ou em serviço.

- **Vantagem:**
  - **Escalonamento automático** de ISRs com base na prioridade de hardware.
  - Torna o gerenciamento complexo de interrupções mais simples no software.

ISR (*Interrupt Service Routine*) e Prioridade

INSTITUTO MAUÁ DE TECNOLOGIA
MAUÁ

- **Definição de ISR:**
  - Rotina de serviço de interrupção. É o **código que roda** assim que a interrupção ocorre.

- **Características Importantes de ISRs:**
  1. **Devem ser curtas e objetivas:** Evitar loops longos ou uso de funções bloqueantes (ex.: `delay()` ou `printf()` grandes).
  2. **Variáveis voláteis:** Se a ISR altera variáveis usadas no loop principal, declarar como `*volatile*` ou usar mecanismos de sincronização (ex.: semáforos).
  3. **Interrompibilidade:** Uma ISR pode ser interrompida se chegar outra interrupção de prioridade mais alta (NVIC).

- **Prioridade:**
  - Determinada via registros do NVIC.
  - Tarefas críticas → prioridade mais alta.
  - Tarefas menos urgentes → prioridade mais baixa.

Lista de interrupções do RP2040

- **Onde encontrar:**
  - Datasheet oficial do RP2040 → Tabela de Vetores de Interrupção.
  - Inclui interrupções para: **GPIO** (pinos agrupados ou individuais), **TIMER** (timer do sistema), **UART**, **SPI**, **I2C**, **USB**, **DMA** (*Direct Memory Access*), **Watchdog**, **PIO**, e outros.

- **Exemplo de algumas interrupções comuns:**
  - **GPIO IRQ 0, 1, 2, 3** : Dependendo do agrupamento, configura qual pino dispara qual IRQ.
  - **SysTick Handler** : Timer especial do ARM (se implementado), geralmente usado para RTOS ou tarefas temporizadas.
  - **Timer IRQ** : Interrupções de timer específicas do RP2040.

Interrupção de GPIO: Conceitos e Configuração

- **Como funciona no RP2040:**
  - Cada GPIO pode disparar interrupção em ***Rising Edge***, ***Falling Edge***, ***High Level*** ou ***Low Level***.
  - As interrupções de GPIO podem ser roteadas para até 4 controladores de IRQ (GPIO IRQ 0..3).
  - No SDK C/C++ do Pico, há funções utilitárias como `gpio_set_irq_enabled()` e `gpio_set_irq_enabled_with_callback()`.

- **Borda vs Nível:**
  - **Borda (*Rising*/*Falling*)**: Interrupção ocorre quando o sinal passa de 0→1 ou 1→0.
  - **Nível (*High*/*Low*)**: Interrupção permanece ativa enquanto o sinal estiver em nível alto ou baixo. (Menos comum para botões, mas útil em outras aplicações.)

- **Configuração típica para um botão:**
  - ***Pull-up*** ou ***Pull-down*** interno.
  - Interrupção em borda de descida ou subida (dependendo da lógica do hardware).
  - Tratar *debounce* (mecânico) por hardware (capacitor) ou software.

Passos Detalhados para Configurar Interrupções de GPIO no RP2040

INSTITUTO MAUÁ DE TECNOLOGIA
MAUÁ

- **Para configurar interrupções no RP2040 (usando o Pico SDK em C/C++), seguem os passos:**
  1. **Escolha do pino** e verifique o hardware (botão, resistor de *pull-up*/*pull-down*).
  2. **Inicialize o pino** no código, definindo direção e *pull-up* ou *pull-down*:
  3. **Selecione o tipo de evento** que disparará a interrupção (*RISING*, *FALLING*, *HIGH*, *LOW*).
  4. **Habilite a interrupção** e registre a função de *callback* que tratará o evento:
  5. **Escreva a função de *callback* (ISR).**
  6. **Trate *Debounce* (se necessário)**: Em um botão físico, considere ruídos de contato.
  7. **Mantenha a ISR curta** : Evite funções bloqueantes e variáveis que não sejam atômicas.

- **Como o SDK faz a "mágica":**
  – O `gpio_set_irq_enabled_with_callback()` internamente configura o hardware do RP2040 para direcionar a interrupção ao **NVIC**, e registra a função `gpio_callback` como ISR para o conjunto de pinos configurados.

Usos de Interrupção de GPIO (Botão, *Encoder*, etc.)

- **Botão:**
  - Clássico “pressionar para acender/apagar LED” ou iniciar uma ação (ex.: envio de dados).
  - Elimina necessidade de *polling* contínuo.
- **Encoder Rotativo:**
  - Cada clique ou mudança gera pulsos em dois sinais A/B.
  - Velocidade de rotação pode ser alta, demandando resposta rápida.
  - Interrupções evitam perda de pulsos.
- **Sensores de pulso:**
  - Leitura de contador de eventos.
  - **Exemplo:** sensor de efeito Hall para medir rotação de motor ou fluxo de água.
- **Portas de expansão:**
  - Vários dispositivos externos sinalizando eventos (ex.: um teclado matricial, com lógica que gera interrupção ao pressionar uma tecla).

Uso de Interrupções em Modos de Baixo Consumo

- **Por que baixo consumo?**
  - Em aplicações IoT ou alimentação por bateria, o tempo em modo ativo precisa ser reduzido.
  - O RP2040 pode **desligar alguns *clocks*** e entrar em modos de ***sleep***.

- **Papéis das interrupções:**
  - Permitem que o MCU durma e acorde somente ao receber um evento importante (ex.: um botão pressionado, sensor detectando movimento).

- **Como funciona:**
  - Geralmente, chamamos instruções como `__wfi()` (*Wait For Interrupt*) ou `__wfe()` (*Wait For Event*).
  - NVIC permanece “escutando” se alguma fonte de interrupção for habilitada.
  - Ao ocorrer a interrupção, o core **retoma** a execução imediatamente após a instrução de *sleep*.

Modos de Baixo Consumo do RP2040

- **Modos disponíveis (segundo datasheet)**
  - ***Dormant*** (modo “adormecido” mais profundo).
  - ***Sleep*** e variações, onde alguns blocos de *clock* podem ser desativados.
  - Redução de frequência de *clock* (ex.: 133 MHz → menor *clock*).

- **Configurando no SDK:**
  - Use `sleep_run_from_xosc()` ou similares (conforme o *clock* pretendido).
  - Para modo ***sleep***, podemos chamar `__wfi()` no firmware.
  - Verificar se há **periféricos em operação** que precisam permanecer ativos (ex.: RTC, alarme, etc.).

- **Como acordar o RP2040:**
  - **Fonte de interrupção** configurada no NVIC.
  - Pode ser GPIO, Timer, UART (receber dado), etc.
  - Precisamos habilitar a interrupção antes de entrar em *sleep*.

Detalhes sobre Modo *Sleep* e Por que `__wfi()` Acorda o RP2040

- **Entrada em modo *Sleep***
  - A instrução `__wfi()` (*Wait For Interrupt*) é o método mais simples para reduzir consumo em um Cortex-M.
  - Ao chamá-la, o **núcleo** (core) do processador entra em estado de espera, enquanto o NVIC continua monitorando interrupções habilitadas.

- **O que é necessário configurar antes?**
  - **Interrupção habilitada** : Se deseja acordar via GPIO, é preciso configurar esse GPIO para gerar interrupção e habilitá-la no NVIC (o SDK faz isso automaticamente ao usar `gpio_set_irq_enabled_with_callback`).
  - **Estado do pino**: Certifique-se de usar *pull-up*/*pull-down* corretos para que a borda aconteça de forma previsível.

Detalhes sobre Modo *Sleep* e Por que `__wfi()` Acorda o RP2040

- **Despertar ao receber interrupção**
  - Quando o GPIO (ou outro periférico) dispara uma interrupção, o NVIC reconhece o evento e **derruba** o estado de *sleep*.
  - A CPU retoma a execução imediatamente após a instrução `__wfi()`, executando a ISR correspondente.

- **Modos mais profundos**
  - Há modos que desligam mais partes do chip (por exemplo, "*Dormant mode*"). Porém, em muitos casos, `__wfi()` já traz consumo bastante reduzido, especialmente se você desativar módulos e reduzir o *clock* previamente.

- **Observação**
  - Se a interrupção não estiver habilitada ou estiver configurada para borda errada (ex.: *FALL* em vez de *RISE*), o RP2040 não acordará.
  - Sempre revise a configuração do pino e do evento de interrupção.

Exemplo 1: Interrupção de GPIO (Botão + LED)

- **Objetivo:**
  - Ao pressionar o botão, alternar (*toggle*) o LED *onboard* do Raspberry Pi Pico.

- **Passos principais:**
  1. Conexão do botão a um GPIO (ex.: GPIO 15) com resistor de *pull-up* ou *pull-down*.
  2. Configurar o GPIO como entrada no SDK.
  3. Habilitar interrupção na borda desejada (p. ex. *Falling Edge*).
  4. Escrever a função de *callback* (ISR) que alterna o estado do LED.
  5. No `main()`, manter o loop ocioso, pois a ação ocorrerá na ISR.

- **Pontos de atenção:**
  - Cuidado com ***debounce*** do botão (*hardware* ou *software*).
  - Não realizar tarefas demoradas na ISR.

Exemplo 2: Sleep Mode acordado por Botão

- **Objetivo:**
  - Demonstrar como colocar o RP2040 em modo de baixa energia e acordar com um botão.

- **Passos principais:**
  1. Configurar um GPIO como entrada e habilitar interrupção (borda de descida, por exemplo).
  2. No código, entrar em **modo *sleep*** usando `__wfi()`.
  3. Ao pressionar o botão, a interrupção dispara e o RP2040 acorda, retornando a execução ao ponto após `__wfi()`.

- **Pontos de atenção:**
  - Em aplicações reais, você configuraria *clocks* para reduzir consumo também (por ex., dormindo com oscilador desligado).
  - Poderia usar outro GPIO para sinalizar atividade ou reacender o LED.

Exemplo 2: Sleep Mode acordado por Botão

- **Objetivo:**
  - Demonstrar como colocar o RP2040 em modo de baixa energia e acordar com um botão.

- **Passos principais:**
  1. Configurar um GPIO como entrada e habilitar interrupção (borda de descida, por exemplo).
  2. No código, entrar em **modo *sleep*** usando `__wfi()`.
  3. Ao pressionar o botão, a interrupção dispara e o RP2040 acorda, retornando a execução ao ponto após `__wfi()`.

- **Pontos de atenção:**
  - Em aplicações reais, você configuraria *clocks* para reduzir consumo também (por ex., dormindo com oscilador desligado).
  - Poderia usar outro GPIO para sinalizar atividade ou reacender o LED.